

PROFILE



정관우

Backend Developer

연락처 010-3361-1654
이메일 jkw4805@naver.com
GitHub <https://github.com/JeongGwanWoo>

Main Stack

언어 Java
서버 Spring Boot
DB MySQL / PostgreSQL
인프라 Redis / Docker / Jenkins

Focus

API

DB

상태 흐름

서비스 검증

서비스 흐름을 안정적으로 연결하는 백엔드 개발자

사용자의 요청이 저장, 처리, 결과 확인까지 이어지는 과정을 데이터와 상태 기준으로 이해하고 구현합니다.

{ Education }

- 2025.07 ~
삼성청년SW-AI아카데미(SSAFY)
SW-AI 실무 중심 개발자 교육 과정
- 2016.03 ~ 2022.02
동신대학교
디지털 콘텐츠 학과

{ Certifications }

- 2026.06 120 (IM2)
TOEIC® Speaking
- 2021.08
정보처리기사

{ Projects }

- 2026.04 ~ 2026.05
Wedge
브라우저 실행 기반 UX 진단 리포트 서비스
- 2026.03 ~ 2026.04
DonDone
외국인 근로자 급여 지급 데모 MVP
- 2026.01 ~ 2026.02
당낭콩(DNC)
CGM 활동 데이터 기반 헬스케어 서비스
- 2025.12
MyTripQuest
위치 기반 퀘스트와 AI 사진 인증 여행 서비스

Wedge

브라우저 실행 기반 UX 진단 리포트 서비스

개발 기간 2026.04 ~ 2026.05 약 7주

플랫폼 Web

개발 인원 6명

담당 역할 Spring Backend

- 분석 진행 상태 연동
- Analysis Outbox/Retry
- Analyzer 병렬 처리 검증

개발 환경

언어 Java 17 / TypeScript / Python

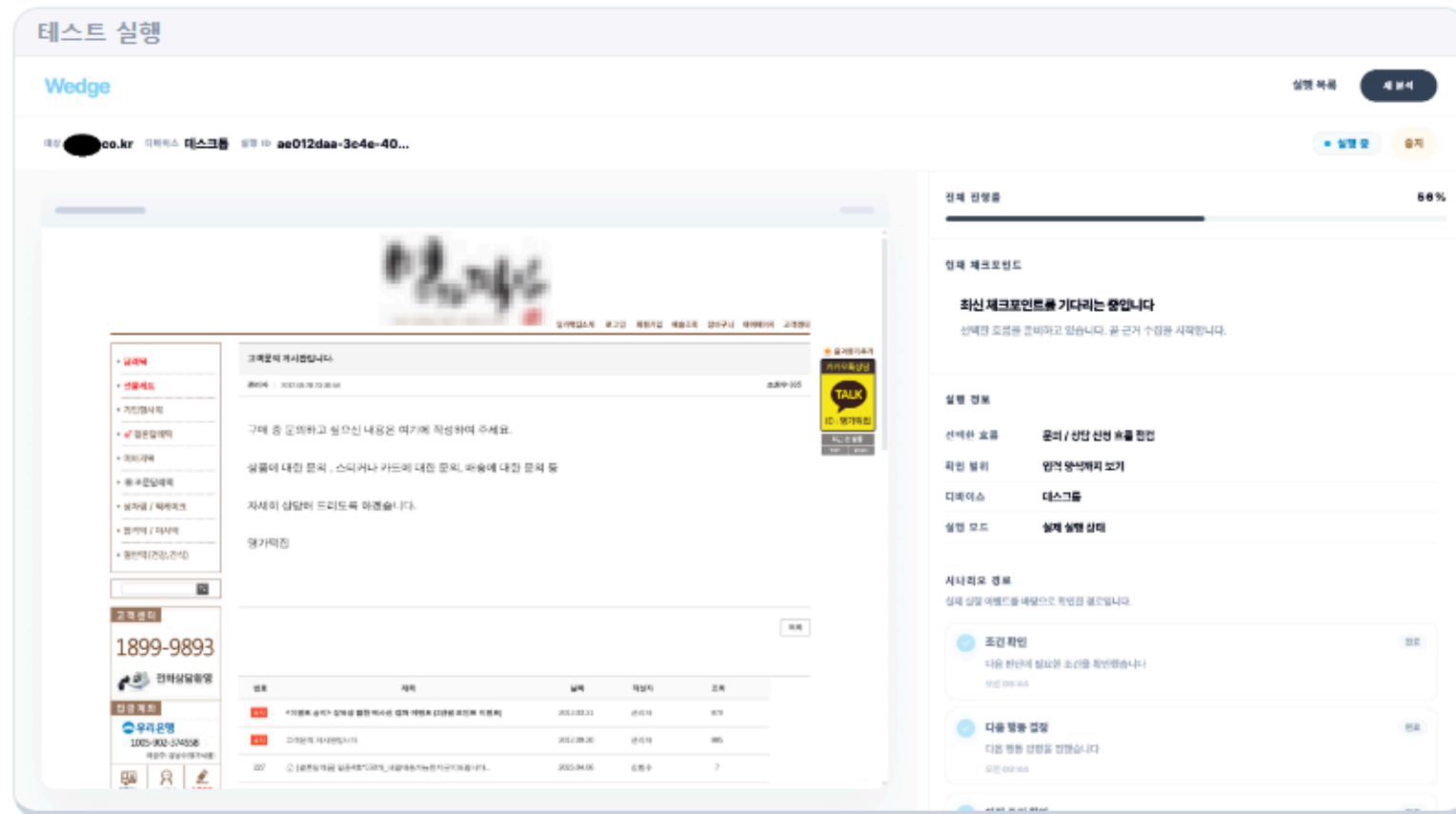
서버 Spring Boot 3.5.14 / MyBatis

프론트 React / Vite

Runner Node.js 24 / Playwright

DB / MQ PostgreSQL 17 / Redis 8.6 / RabbitMQ

인프라 Docker / Nginx / Jenkins / S3



{

세부 구현사항 1

분석 진행 상태 연동

- 브라우저 실행기가 보내는 진행 결과 알림을 Spring 서버에서 수신했습니다.
- 실행 단계별 상태를 DB 기준 상태로 반영했습니다.
- 사용자가 보는 진행 상황과 실제 처리 상태가 어긋나지 않도록 연결했습니다.

Analysis Outbox/Retry

- 분석 Job 상태 변경과 outbox 메시지 저장을 하나의 DB 트랜잭션 안에서 처리했습니다.
- MQ 발행은 트랜잭션 커밋 이후 실행되도록 분리해, 커밋 전 발행되는 상황을 피했습니다.
- 발행 실패 시 outbox에 실패 상태와 다음 시도 시점을 남겨 재전달 가능한 흐름으로 구성했습니다.

}

분석 요청이 처리되는 흐름

사용자의 분석 요청이 저장되고, 작업 큐를 거쳐 브라우저 실행·분석 처리 결과가 다시 상태로 반영되는 구조입니다.

사용자 분석 요청

웹 화면에서 URL과 분석 조건 입력



Spring API Server

요청 저장 · 실행 상태 생성



Analysis Outbox

전달 대기 작업 기록 · 재시도 기준



작업 큐

브라우저 실행기와 분석 처리기로 작업 전달



브라우저 실행 · 분석 처리

화면 실행, 캡처, 판단 결과 생성



진행 상태 업데이트

진행/완료/실패 상태를 서버 DB에 반영

- 핵심은 요청 저장, 작업 전달, 진행 상태 반영이 중간에 끊기지 않도록 연결한 것입니다.

{

세부 구현사항 2

Analyzer 병렬 처리 적용

- 리포트 생성 중 오래 걸리는 이미지 판단 요청을 일부 병렬로 처리했습니다.
- 한 번에 너무 많은 요청이 몰리지 않도록 동시 처리 수를 제한했습니다.
- 처리 방식별 실행 시간을 기록해 개선 과정을 비교했습니다.

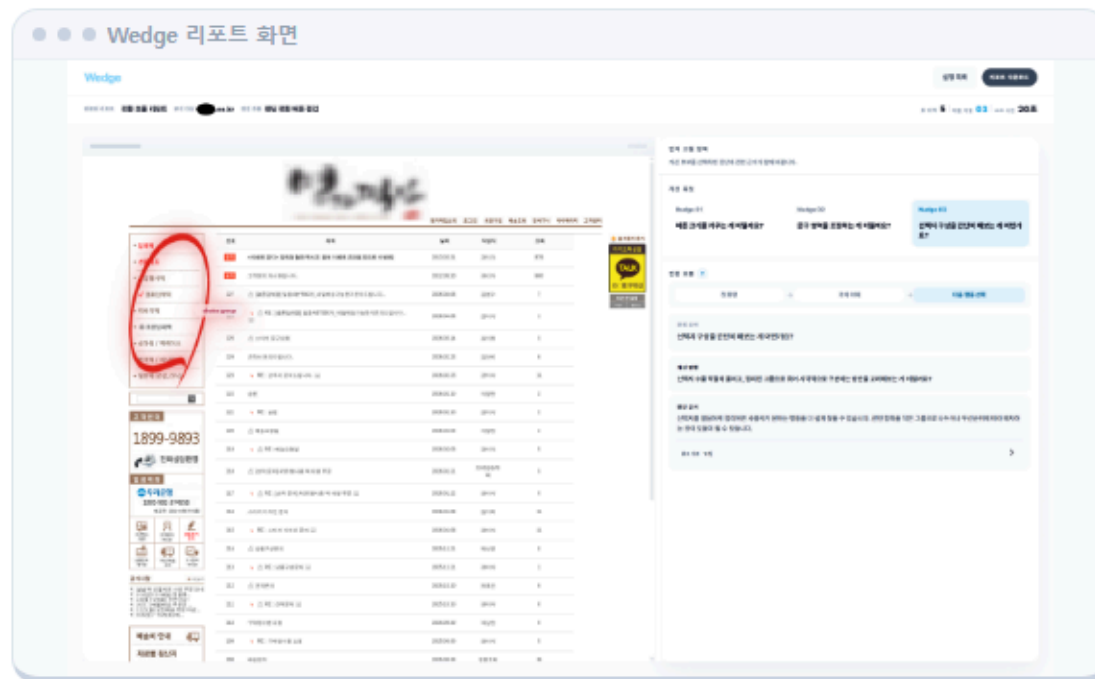
결과 안정성 검증

- 순차 처리와 병렬 처리의 판단 결과가 같은지 비교했습니다.
- 속도 개선이 결과 품질을 해치지 않는지 확인했습니다.
- 실패 요청이 숨겨지지 않도록 예외 흐름과 측정 조건을 함께 남겼습니다.

}

리포트 생성 대기 시간 개선

사용자가 분석 완료를 기다리는 시간을 줄이기 위해, 오래 걸리는 이미지 판단 구간을 병렬 처리하고 결과 동일성을 확인했습니다.



/ 분석 결과 화면 / 리포트 생성 완료

첫 번째 개선

84.0s → 63.5s

화면 3개를 일부 동시에 확인
약 24.4% 단축

두 번째 개선

63.5s → 42.3s

63.5초 기준 환산
약 33.4% 단축

{

문제 해결과 회고

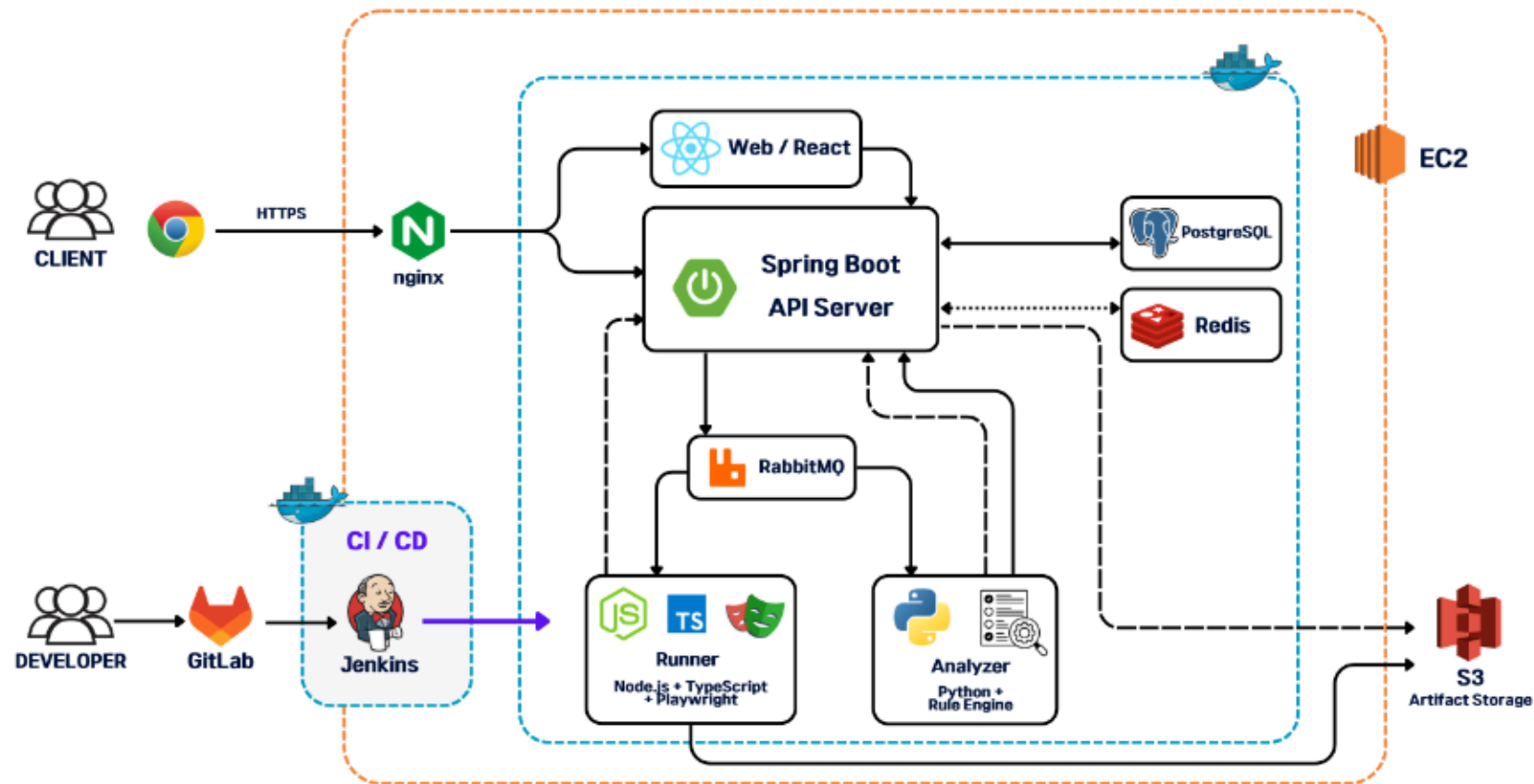
문제 해결 관점

- 분석 상태를 대기 상태로 바꾼 뒤 MQ 발행이 실패하면, 화면 상태와 실제 작업 전달 상태가 어긋날 수 있었습니다.
- 상태 변경과 **outbox 저장**은 DB 트랜잭션으로 함께 처리하고, MQ 발행은 커밋 이후 단계로 분리했습니다.
- 발행 실패 기록과 재시도 기준을 남겨 작업이 조용히 사라지지 않도록 보완했습니다.
- **Analyzer** 병렬 처리 개선은 속도 수치뿐 아니라 결과 동일성과 실패 흐름을 함께 확인했습니다.

프로젝트 회고

역할이 나뉜 서비스에서는 각 영역이 주고받는 데이터 기준을 명확히 해야 변경 리스크를 줄일 수 있었습니다. 또한 사용자가 보는 진행 흐름과 서버 내부 상태가 어긋나지 않도록 맞추는 과정이 백엔드 구현의 핵심이라는 점을 배웠습니다.

}



DonDone

외국인 근로자를 위한 근무 기록 기반 급여 지급

개발 기간 2026.03

플랫폼 Mobile App / 고용주 관리 화면

프로젝트 외국인 근로자 급여 지급 demo MVP

담당 역할 Spring 백엔드

- 출퇴근 기록과 위치 검증 흐름
- 급여 결과 저장·문서/문제 제기 연결
- 고용주 검토·미리받기 일부

개발 환경

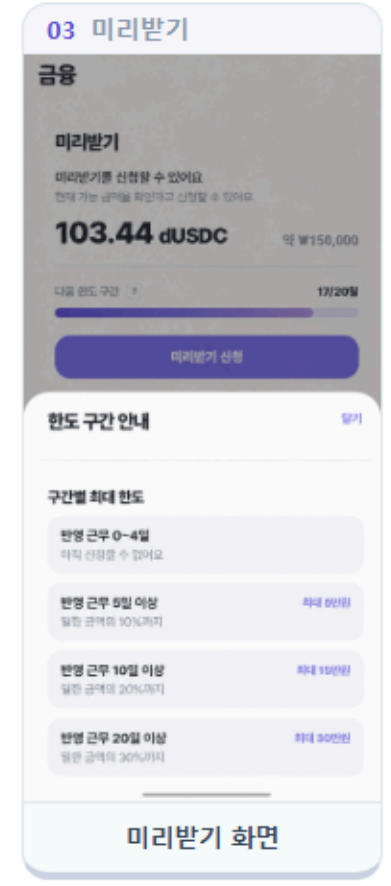
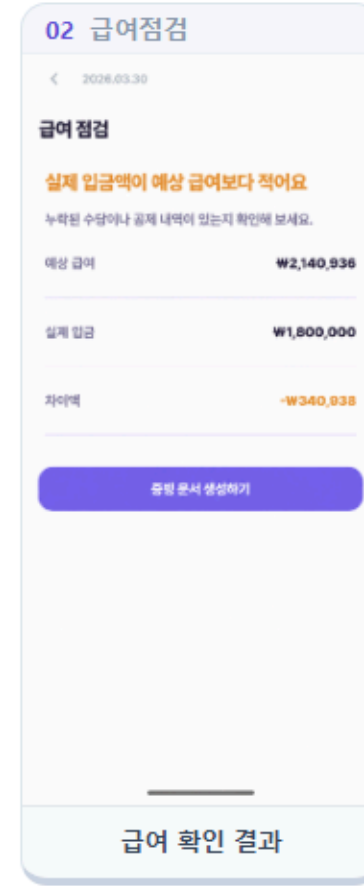
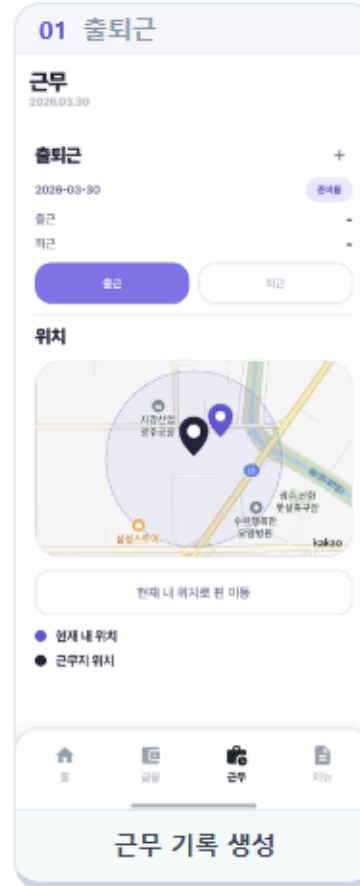
언어 Java 17

서버 Spring Boot 3.2 / Spring Security

DB PostgreSQL / JPA / Querydsl

문서 OpenAPI / Thymeleaf / OpenHTMLtoPDF

검증 관련 통합·단위 테스트 근거



{

세부 구현사항 1

출퇴근 기록과 위치 검증 흐름

- 출근 기록은 근무지 반경 밖에서 요청되면 등록되지 않도록 처리했습니다.
- 퇴근 기록은 반경 밖에서 요청되어도 바로 삭제하지 않고 우선 기록으로 남겼습니다.
- 대신 해당 기록은 검토 필요 상태로 분리했습니다.
- 고용주 확인을 거친 기록만 급여 확인과 미리받기 계산에 반영되도록 했습니다.

문제 해결 관점

- 무조건 차단하면 실제 근무 기록이 사라질 수 있었습니다.
- 무조건 반영하면 부정확한 기록이 급여 확인과 미리받기 계산에 포함될 수 있었습니다.
- 따라서 기록 보존과 계산 반영 기준을 분리했습니다.

}

근무지 밖 퇴근 기록 처리 흐름

근무지 밖 퇴근 기록을 바로 삭제하지 않고, 검토 후 계산에 반영하는 흐름입니다.

출근 기록

근무지 위치 확인
반경 밖이면 등록 차단

정상 퇴근

기록 저장 후 계산 반영 후보



반경 밖 퇴근

기록은 보존하고 검토 대상으로 분리

검토 필요 → 계산 반영

고용주 확인 후 급여/미리받기 계산에 반영

{ 세부 구현사항 2

급여 계산 기준 스냅샷

- 월간 출퇴근 기록을 기준으로 급여 확인 결과를 생성했습니다.
- 실제 임금액과 예상 급여 차이를 **급여 확인 시점의 스냅샷**으로 저장했습니다.
- 이후 문서 생성과 문제 제기 준비에서 같은 급여 확인 결과를 참조하도록 했습니다.

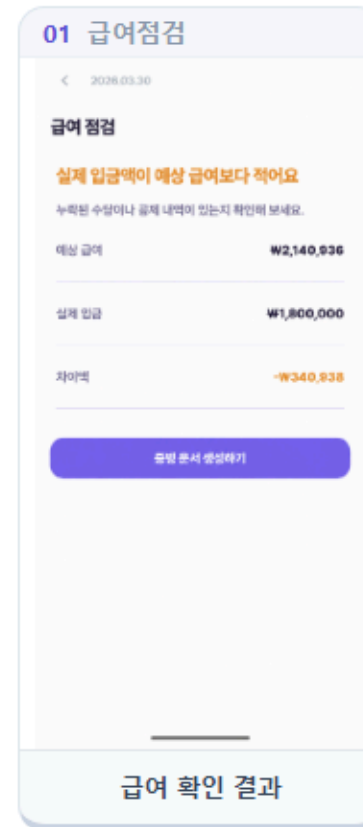
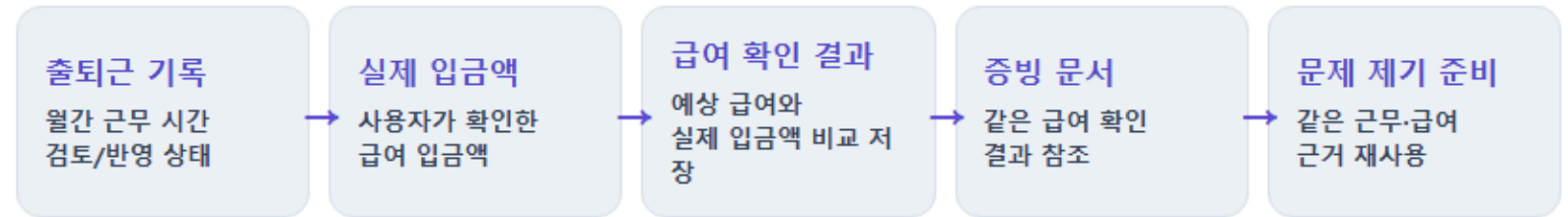
증빙 문서 / 문제 제기 기준 연결

- 증빙 문서와 문제 제기 자료를 만들 때, 해당 급여 확인 결과가 사용자의 것인지 확인했습니다.
- 문제 제기 준비 화면에서도 같은 급여 확인 결과를 재사용했습니다.
- 문서 생성 흐름에서는 중복 요청 처리 근거가 확인되어, 해당 범위로만 표현합니다.

}

같은 근무 기록을 여러 기능에서 재사용하는 흐름

급여 확인 결과를 매번 다시 계산하지 않고 저장해, 증빙 문서와 문제 제기 준비가 같은 기준을 보도록 연결했습니다.



{

검토·승인 흐름과 테스트 보강

고용주 등록과 검토 흐름

- 고용주가 발급한 등록 코드를 근로자가 입력해 회사/사업장에 연결되는 흐름을 구현했습니다.
- 근로자와 회사/사업장 연결 정보를 만들고, 검토가 필요한 기록을 고용주가 확인할 수 있는 목록으로 구성했습니다.
- 고용주 확인 후 '검토 필요' 기록을 '계산 반영' 상태로 전환했습니다.

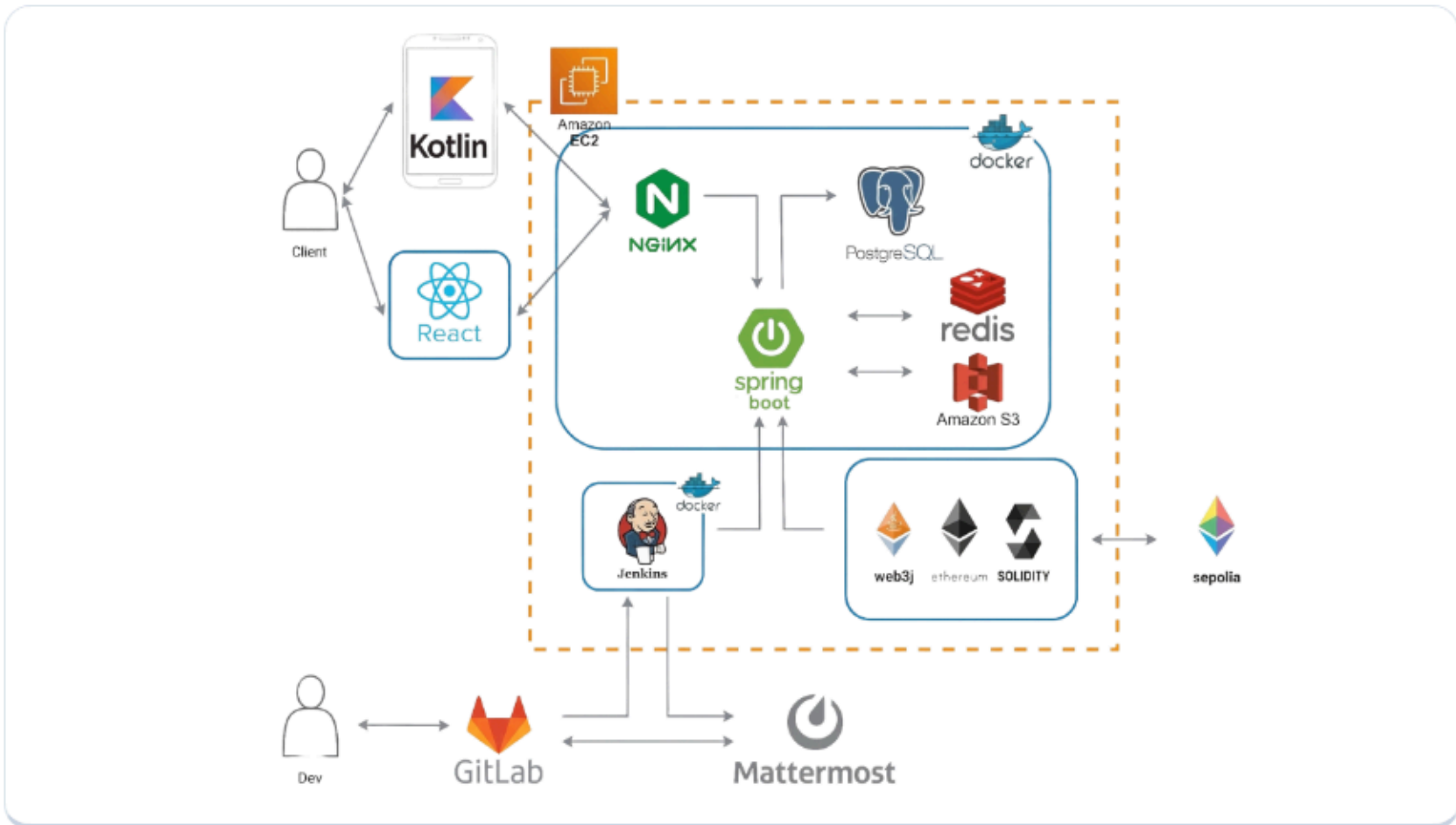
미리받기 관리자 승인

- 관리자가 미리받기 요청을 승인할 수 있는 API를 구현했습니다.
- 인정된 근무 시간이 미리받기 가능 여부 계산에 반영되는지 회귀 테스트를 보강했습니다.
- 개인 기여는 관리자 승인 API와 인정 시간 회귀 테스트 범위로 제한해 표현합니다.

프로젝트 회고

단순히 기록을 저장하는 것보다, 어떤 기록을 계산에 반영할 수 있는지 상태를 분리하는 것이 중요했습니다. 하나의 출퇴근 기록이 급여 확인, 증빙 문서, 문제 제기, 미리받기 판단의 기준이 되기 때문에 기준 데이터를 명확히 고정하는 경험을 했습니다.

}



당낭콩(DNC)

AI 기반 개인화 혈당 예측·식단
코칭 헬스케어 서비스

개발 기간 2026.01 ~ 2026.02

플랫폼 Mobile App

개발 인원 팀 프로젝트

- 담당 역할 Backend + Infra
- 혈당 데이터 저장 흐름
 - AI 식단·리포트 연결
 - 승인 배포/결과 공유 흐름

개발 환경

언어 Java 17 / Python / TypeScript

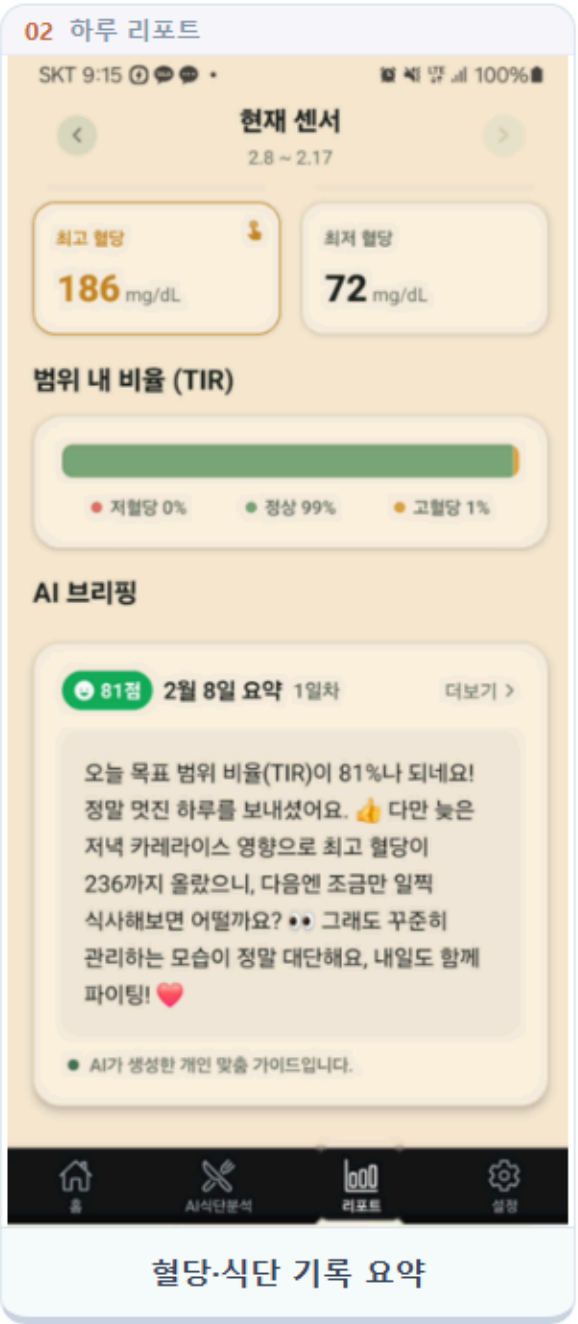
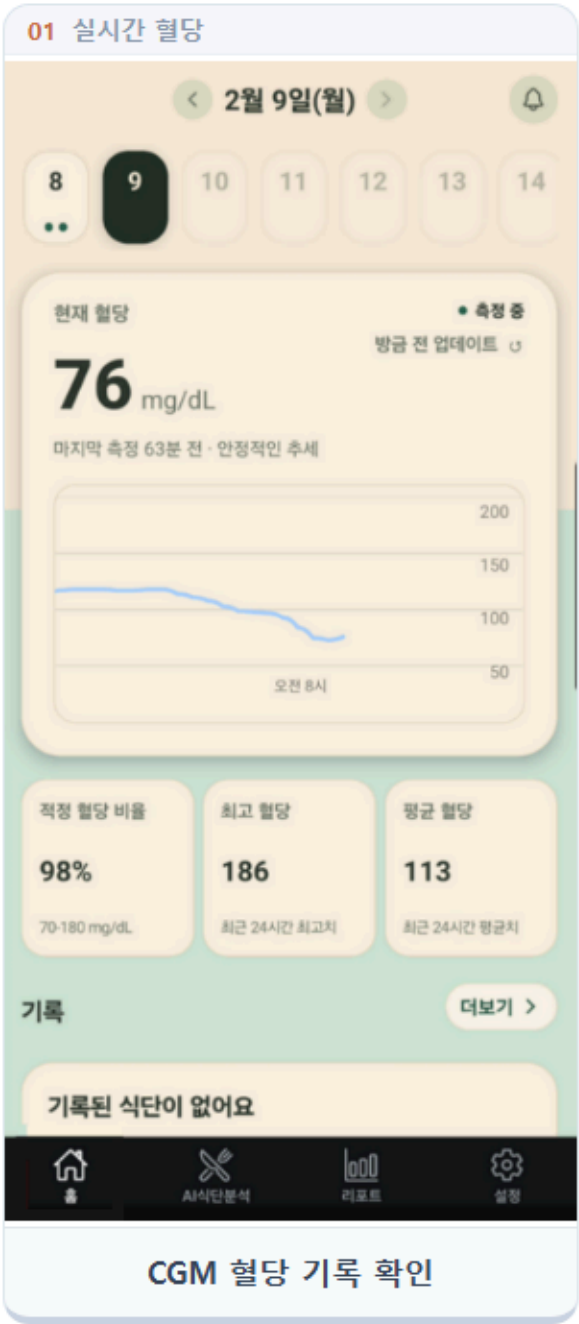
서버 Spring Boot 3.5.9 / FastAPI

모바일 React Native / Expo

DB/Cache MySQL / Redis

외부연동 Dexcom OAuth / Gemini / FCM / S3

인프라 Docker / Jenkins / Nginx



{

세부 구현사항 1

외부 혈당 데이터 수집

- 외부 혈당 측정 서비스에서 **5분마다** 새 혈당 기록을 가져왔습니다.
- 인증이 만료되면 갱신 후 다시 요청하고, 가져온 데이터는 임시 저장 공간을 거쳐 DB에 저장했습니다.

사용자별 혈당 기록 분리

- 사용자별 임시 저장 공간을 분리해 혈당 기록이 섞이지 않도록 했습니다.
- 같은 혈당 기록이 반복 저장되지 않도록 사용자와 외부 기록 ID를 함께 확인했습니다.

센서 상태 전환

- 연동 직후에는 센서를 대기 상태로 관리했습니다.
- 첫 혈당 데이터가 들어오면 실제 센서 정보를 연결해 사용 가능 상태로 전환했습니다.

}

외부 혈당 데이터가 사용자 기록이 되는 흐름

외부 혈당 데이터를 그대로 저장하지 않고, 사용자별 임시 저장 공간·중복 기록 확인·센서 상태 전환 기준을 두어 서비스 기록으로 연결했습니다.

외부 혈당 측정 서비스
5분마다 새 혈당 기록 제공



주기적 수집 작업
사용자별 혈당 기록 가져오기



인증 갱신 후 재요청
권한 만료 시 새 인증으로 다시 호출



사용자별 임시 저장 공간
Redis에 사용자 ID 기준으로 분리



중복 기록 확인
사용자와 외부 기록 ID를 함께 확인



DB 저장 / 센서 상태 변경
혈당 저장, 대기 → 사용 가능 전환

- 핵심은 **사용자별 저장 기준**과 **중복 기록 확인 기준**을 명확히 한 것입니다. 실제 장애 감소 수치는 확인되지 않아 정량 성과로 쓰지 않습니다.

{

세부 구현사항 2

식단 분석 결과 연결

- 식단 사진을 음식 분석 서버로 보내 음식명과 양 정보를 받았습니다.
- 분석 결과를 음식 정보와 매칭해 식단 기록과 영양 계산에 사용할 수 있도록 변환했습니다.

예상 혈당 저장

- 섭취량 단계를 계산 가능한 음식 양 비율로 변환했습니다.
- 사용자 정보, 최근 혈당, 식단 정보를 조합해 예상 혈당 결과를 저장했습니다.

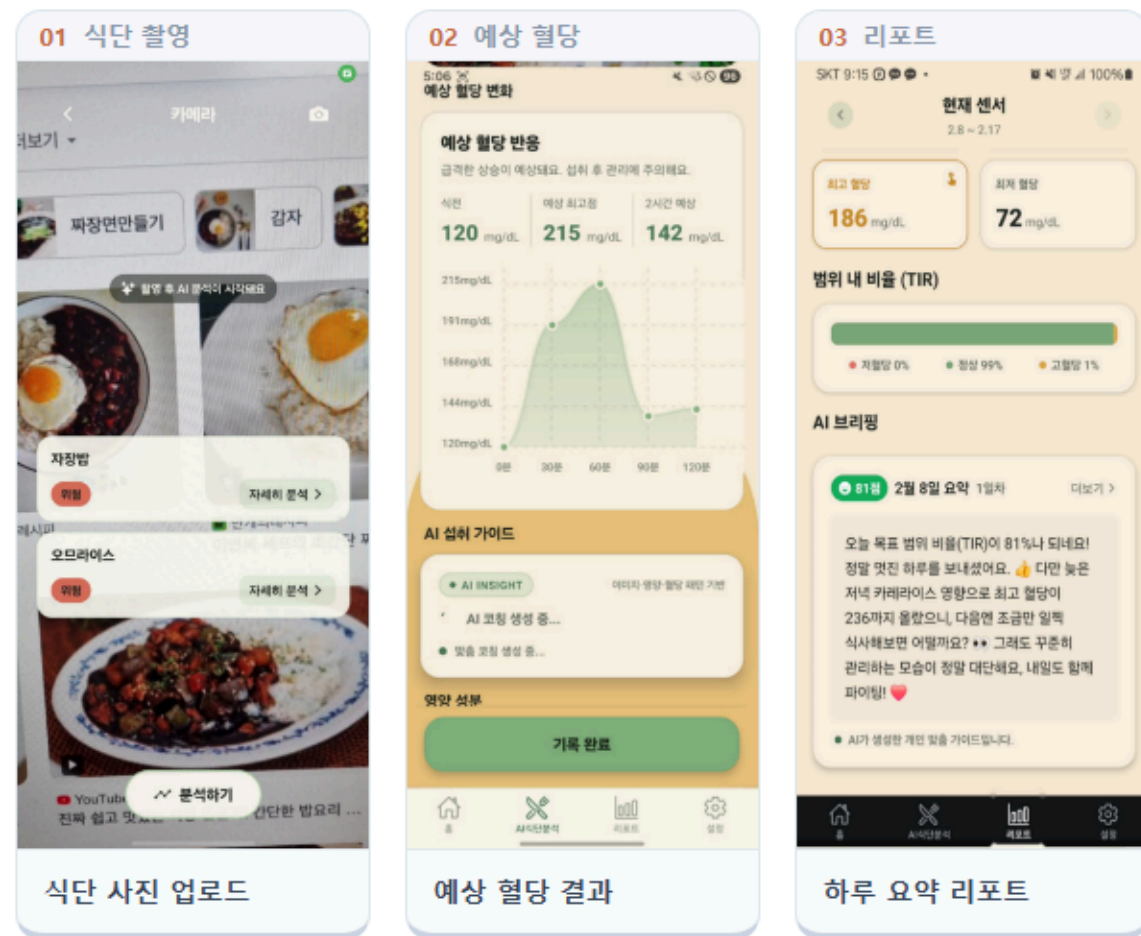
AI 요약과 기본 안내

- 요약 문구가 과장되거나 없는 정보를 추정하지 않도록 출력 조건을 제한했습니다.
- 데이터 부족이나 외부 호출 실패 시 기본 안내 문구를 보여주도록 처리했습니다.

}

AI 분석 결과를 사용자 기록으로 변환

AI 모델 성능이 아니라, 음식 분석 서버의 결과를 식단 기록·예상 혈당·요약 리포트로 연결한 백엔드 흐름에 집중했습니다.



백엔드 처리 흐름

앱 요청 수신

식단 사진 업로드와 분석 요청 처리

이미지 저장 / 음식 분석

저장소 보관 후 음식 분석 서버 호출

식단 기록 변환

음식 정보 매칭, 양 보정, 예상 혈당 저장

요약 문구 생성

출력 조건 제한과 실패 시 기본 안내

{

문제 해결과 운영 흐름

문제 해결 관점

- 혈당 데이터는 사용자별 저장 기준과 중복 확인 기준을 두어 섞이지 않도록 처리했습니다.
- 음식 분석 결과는 식단 기록, 영양 정보, 예상 혈당, 요약 문구로 변환했습니다.
- 외부 호출 실패나 데이터 부족 상황에서는 기본 안내 문구를 제공하도록 처리했습니다.

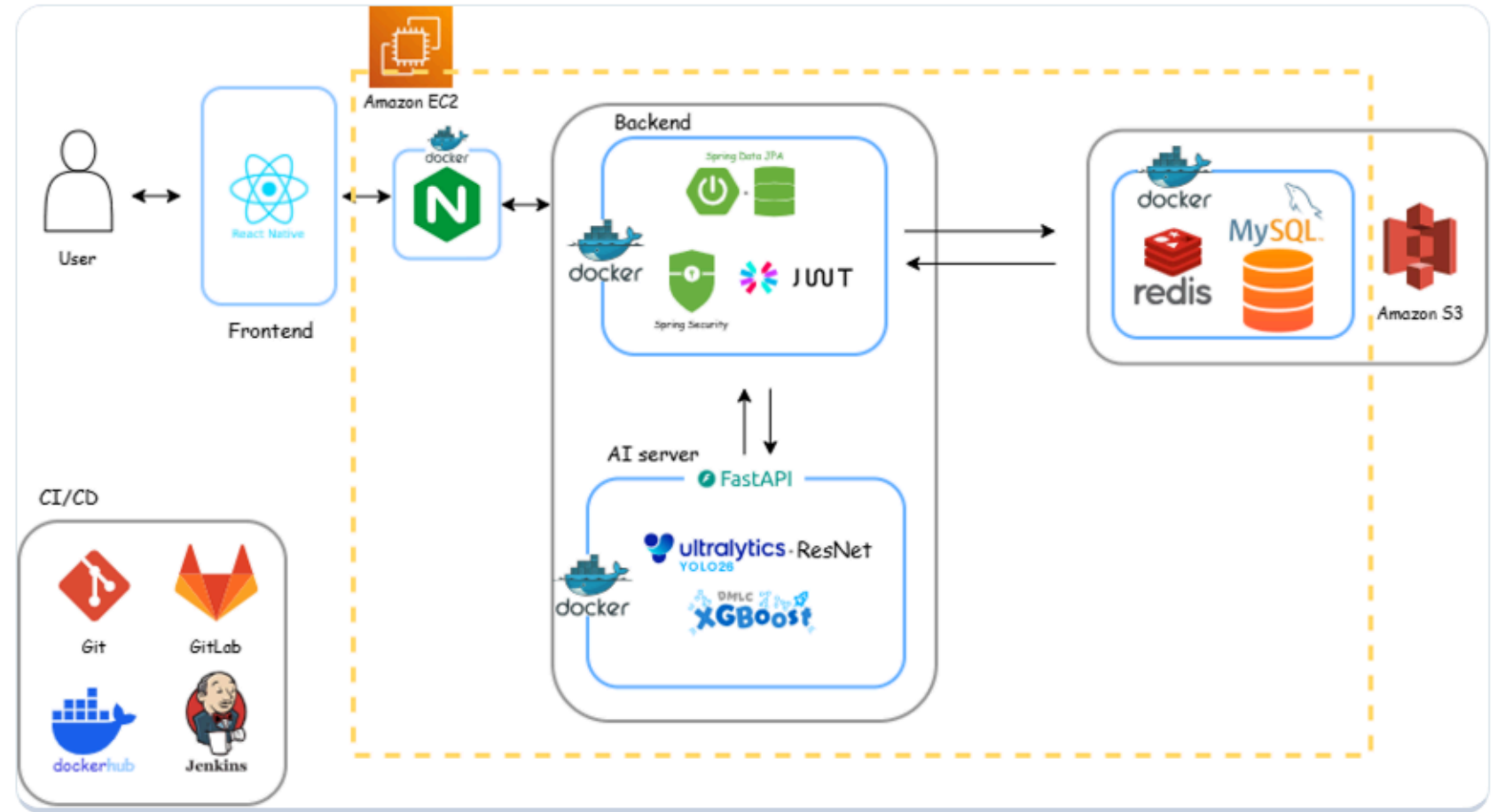
인프라/협업 흐름

- 서비스별 변경 여부를 확인한 뒤 승인 단계를 거쳐 배포하도록 구성했습니다.
- 컨테이너 기반 배포 후 결과를 팀 채널에 공유하는 흐름 구성에 참여했습니다.
- **Backend, AI Server, Mobile**이 함께 배포되는 구조에서 변경 범위와 결과 공유를 맞췄습니다.

프로젝트 회고

외부 서비스와 AI 서버를 연결할 때는 단순 호출보다 사용자별 저장 기준, 실패 시 기본 응답, 중복 방지 기준이 중요하다는 점을 배웠습니다.

}



MyTripQuest

위치 기반
여행 퀘스트 플랫폼

개발 기간 2025.12

플랫폼 Web / Mobile Web

개발 인원 2명

담당 역할 Backend / Frontend 구현 참여

- 퀘스트 인증 흐름 구현 참여
- AI 사진 인증·보상 흐름 구현 참여
- 인증/관리자 백엔드 구현 참여

개발 환경

언어 Java 17 / JavaScript

서버 Spring Boot 3.5.8 / MyBatis

프론트 Vue 3 / Vite / Pinia / Axios

DB MySQL

인증 Spring Security / JWT / OAuth2

외부 API TourAPI / Gemini(GMS) API



{

구현 1. 위치·사진 인증 흐름

사용자가 실제 장소에 도착했는지 확인

- 사용자가 퀘스트를 시작하면 개인별 진행 상황을 저장하고, **먼저 완료해야 하는 미션**이 있는지 확인했습니다.
- 사용자 위치와 관광지 위치 사이의 거리를 계산해 **실제 장소 근처에 도착했는지** 판단했습니다.
- 조건을 만족하면 퀘스트를 완료 처리하고 경험치·포인트 지급, 활동 기록까지 이어지도록 연결했습니다.

사진만으로 완료되지 않도록 검증 조건 분리

- 사진 파일이 있는지, 사진에 포함된 위치 정보나 현재 위치가 있는지, 촬영 시간이 유효한지 순서대로 확인했습니다.
- 먼저 장소에 도착한 뒤 촬영했는지, 사진이 해당 관광지 주변에서 찍힌 것인지 검증했습니다.
- 마지막 단계에서 **AI가 사진 속 장소가 목표 관광지와 맞는지** 확인하도록 연결했습니다.

}

사진 업로드만으로 완료하지 않는 검증 흐름

사진을 올렸다는 사실만으로 성공 처리하지 않고, 퀘스트 진행 상태·실제 위치·촬영 시간·AI 장소 확인을 순서대로 거쳐 완료 처리했습니다.

퀘스트 시작

진행 상황 저장 · 먼저 해야 할 미션 확인



실제 도착 확인

현재 위치가 관광지 근처인지 계산



사진 정보 확인

사진 위치 정보와 촬영 시간 확인



위치·시간 조건 확인

도착 후 해당 장소 근처에서 찍은 사진인지 확인



AI 장소 확인

사진 속 장소가 목표 관광지와 맞는지 확인



완료·보상 지급

퀘스트 완료 처리 후 경험치·포인트 지급

{

구현 2. 관광지 데이터 콘텐츠화

공공 관광지 데이터를 퀘스트로 변환

- 공공 관광지 데이터를 가져와 서비스에서 사용할 수 있는 장소 데이터로 변환했습니다.
- 각 장소에 대해 도착 미션과 사진 인증 미션을 만들고, 사용자가 **순서대로 진행**할 수 있도록 연결했습니다.
- 지역별 장소 구분 기준과 장소별 인증 거리 정보를 함께 저장했습니다.

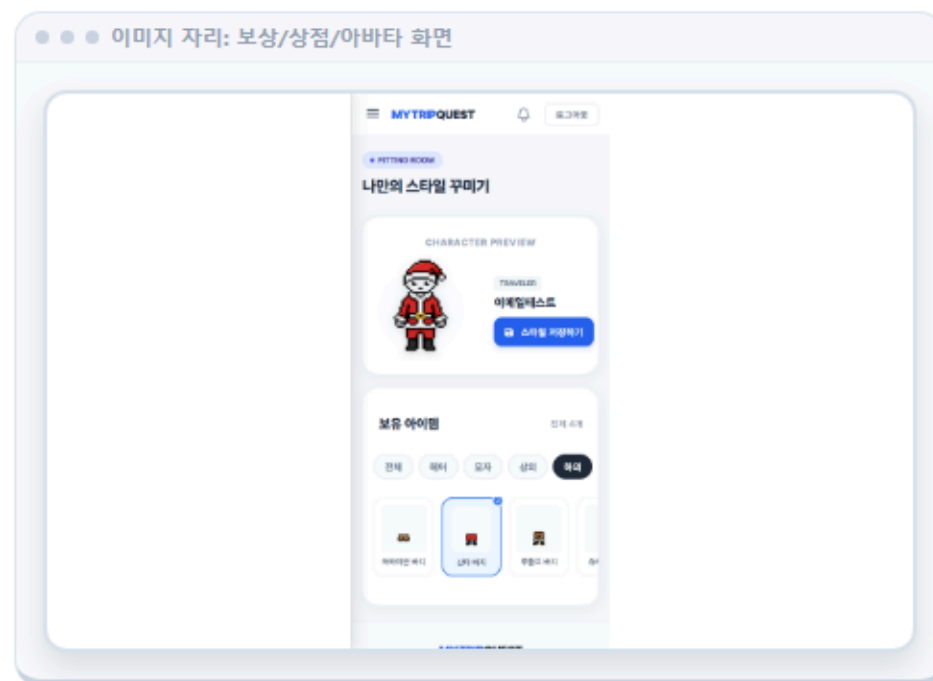
장소별 인증 거리와 보상 흐름 연결

- 관광지명과 주소를 기준으로 장소마다 적절한 인증 거리를 산정하고, 값이 없을 때는 기본값으로 보정했습니다.
- 퀘스트 완료 결과가 경험치·포인트 지급, 상점 구매, 아바타 장착, 랭킹 조회로 이어지도록 연결했습니다.
- 관리자 통계와 시스템 처리 기록은 운영 상황을 확인하는 보조 기능으로 정리할 수 있습니다.

}

관광지 데이터를 서비스 경험으로 전환

공공 관광지 데이터를 그대로 보여주는 데서 끝내지 않고, 사용자가 수행할 수 있는 미션과 보상 흐름으로 연결했습니다.



/ 아바타/상점 보상 흐름

/ 추가 가능: shop.png / rankings.png

관광지 데이터 → 퀘스트 콘텐츠

장소 → 미션

공공 관광지 정보 → 장소 저장 → 인증 거리 설정 → 도착/사진 미션 생성

퀘스트 완료 → 보상 경험

보상 · 아바타 · 랭킹

경험치·포인트 지급 → 상점 구매 → 아바타 꾸미기 → 랭킹 확인

{

문제 해결 관점과 회고

문제 해결 관점

- 사진을 올렸다는 사실만으로는 사용자가 실제 관광지에 방문했는지 확인하기 어려웠습니다.
- AI 사진 확인만으로는 위치, 촬영 시간, 미션 진행 순서를 함께 보장하기 어려웠습니다.
- 외부 서비스와 AI 응답은 예상과 다른 형태로 올 수 있어 예외 상황을 고려해야 했습니다.
- 사용자가 미션 완료 결과를 확인하고 보상을 받는 흐름까지 끊기지 않아야 했습니다.

프로젝트 회고

AI 사진 인증 기능을 구현하며, 사용자 행동 검증은 AI 결과뿐 아니라 위치·시간·진행 상태·보상 처리까지 함께 연결해야 한다는 점을 경험했습니다. 2인 풀스택 프로젝트였기 때문에 공개 자료에서는 팀 성과와 개인 구현 참여 범위를 구분해 표현합니다.

}

